Slack Email Classification Pipeline Architecture

The 10% Heuristic, Kafka Event Streaming & Eventual Consistency

≥ 10%	± Δ Healing
MEMBER DOMAIN THRESHOLD FOR INTERNAL ORG STATUS	RELATIVE DRIFT ADJUSTMENT UNDER AT-LEAST-ONCE DELIVERY

1. The Problem: Enterprise Domain Classification

In Slack's enterprise architecture, onboarding requires dynamically classifying incoming email domains (e.g., `@company.com` vs. `@external.com`):

- **Internal Domains:** Permitted to auto-join workspaces, discover public channels, and invite peers without explicit administrative sign-off.
- **External Domains:** Restricted to guest roles, requiring manual authorization to prevent enterprise data leaks.

2. The 10% Threshold Rule Formulation

Slack codified an empirical classification threshold: if $\ge 10\%$ of provisioned members in a workspace share a specific email domain, that domain is classified as internal:

```
IsInternal(d, w) ⇔ [ C_domain(d, w) / N_total(w) ] ≥ 0.10
```

Computing this ratio synchronously on user registration paths introduces database table scans and unacceptable latency. Slack offloads computations to an asynchronous event pipeline.

3. Event-Driven Architecture with Apache Kafka

User lifecycle events (`user_created`, `role_changed`, `user_deactivated`) are emitted asynchronously to Kafka. Worker consumers process events in the background, updating pre-aggregated domain counter tables and insulating customer-facing API latency.

4. Atomic Database Upserts

To prevent race conditions without costly distributed locks, consumer workers execute atomic SQL upserts:

```
-- Atomic Upsert on Kafka Event Consumption
INSERT INTO workspace_domains (workspace_id, domain, member_count, updated_at)
VALUES (:workspace_id, :domain, 1, NOW())
ON DUPLICATE KEY UPDATE
    member_count = member_count + 1,
    updated_at = NOW();
```

5. The Counter Drift Dilemma

Kafka guarantees **At-Least-Once Delivery**. Under network hiccups or consumer pod crashes, messages are occasionally re-delivered:

The Duplicate Processing Drift Vector

Consumer executes `member_count = member_count + 1`. Network timeout blocks offset commit. Kafka re-delivers event to a new consumer, which executes $+1$ again. Over time, the stored counter drifts away from true database reality ($C_{\text{stored}} \neq C_{\text{true}}$).

6. Why Naive Overwrite Reconciliation Fails

A naive reconciliation job that computes ground truth and executes: `UPDATE workspace_domains SET member_count = :true_count` introduces a fatal concurrency race condition: if an actual user creates an account while the audit query runs, the overwrite stomps over the valid increment, introducing permanent data corruption.

7. The Relative Delta Healer Protocol

Slack resolved the race condition by applying **relative drift deltas ($\pm \Delta$)**:

1. The background healer takes a snapshot of ground truth C_{true} at timestamp T_s.
2. It reads the aggregate table count C_{db} at T_s.
3. It computes the relative drift: $\Delta = C_{\text{true}} - C_{\text{db}}$.
4. It publishes a drift adjustment message to Kafka:

```
UPDATE workspace_domains SET member_count = member_count + (:delta) WHERE domain = :domain
```

Because mutations are relative, concurrent live user signups continue seamlessly alongside the healer, allowing the system to converge to exact consistency.

8. Production Healer Pipeline Telemetry

Pipeline Metric	Synchronous Query Baseline	Slack Eventual Consistency + Healer
Signup Path Latency	280 ms (Table scans)	**18 ms (Asynchronous event emit)**
Counter Accuracy	Exact (Locks tables)	**99.99% (Self-healing within minutes)**
Database Write Contention	High (Row locks on signups)	**Zero contention on user table**

9. Production Architectural Guardrails

1. Threshold Hysteresis to Prevent Rapid Flapping

If a domain hovers around 10% (e.g., 9.9% to 10.1%), classification could flip back and forth. Apply hysteresis buffers (promote at 10.5%, demote at 9.5%) to maintain stable enterprise permissions.

2. Always Reconcile with Relative Math

Never overwrite absolute values in distributed counter tables. Emitting relative signed delta adjustments ($\pm \Delta$) preserves in-flight concurrent writes.